AE3-TL: Faster and More Exact Page-Level Table Extraction with a Real-Time Detection Transformer

APEXE3 labs@apexe3.com Usman Khan usman@apexe3.com

Code: https://github.com/apexe3/ae3-table-extract Weights: https://huggingface.co/APEXE3HQ/ae3-tl

Abstract

Page-level table extraction recovers every table on a document page, together with its grid structure, cell text, caption and footer. On PubTables-v2 [1], our re-run of the previous best model's released weights (POTATR [6]) gets 85% of tables exactly right on pages with up to 20 cells, but only 40% on pages with more than 250 (exact-match columns of Table 7, Appendix C). We present AE3-TL, which pairs the D-FINE real-time detection transformer [11] with a relation head for captions and footers. Starting from an ImageNet [15] backbone, it is trained on table crops and then on full pages, in about 200 H100-hours. Under the official scorer (v0.2.0) [16], AE3-TL improves on POTATR [6] on all six table metrics (GriTS [3], exact match [2] and TEDS [9], each for structure and content), with paired 95% bootstrap intervals [14] above zero. Exact table match rises from 64.5% to 73.7% overall (+9.2 points, 95% CI [8.4, 10.0]), from 85% to 87% on pages with up to 20 cells, and from 40% to 59% on pages with more than 250 cells (density figures from an earlier scoring run of the same weights, before a final word-handling fix). Per-page processing, including post-processing, is 1.73× faster on the same GPU. Code is available at https://github.com/apexe3/ae3-table-extract and weights at https://huggingface.co/APEXE3HQ/ae3-tl.

1 Introduction

Tables carry much of the quantitative content of scientific, financial and regulatory documents. Downstream systems read their values by row and column, so a single structural error, such as a missed row or a merged column, can misplace every value in the rows or columns it touches. Page-level table extraction [1] turns a page image into structured tables: it locates each table, recovers its grid, gives every cell its text and attaches the caption and footer. Its most demanding measure is exact table match: the share of tables reproduced without a single error [2, 3].

Two kinds of system compete on this task. Generative parsers read the page image and write the tables out as markup, recognising the text themselves. In the benchmark's own evaluation [6], their exact content match ranges from 3.0% for SmolDocling [4] and 16.7% for DeepSeek-OCR [5] to 18.7% for dots.ocr [17] and 35.2% for Gemini 3.1 Pro [18]. The last two cost $4,281 and $15,000 per million pages (Appendix B). Detection-based extraction predicts tables, rows, columns, headers and spanning cells as objects, derives the cells from their intersections, and takes the text from the document's word layer when one exists. The previous best model, POTATR [6],[1] takes this route. It extends the Table Transformer (TATR) [2], itself built on the original DETR [7], with a Relationformer-style relation head [8]. With the document's words it reaches 64.5% at $47 per million pages [6]. The two families are not compared like for like, however. With OCR words in place of the word layer, POTATR's exact content match falls below 7%, while its exact topology match, which ignores text, stays at 0.66–0.70 against 0.63 for Gemini [6]. Most of the content gap therefore reflects the text source, and our setting assumes a born-digital page with a word layer. To our knowledge, POTATR is the only model trained and reported on PubTables-v2's page-level task. On cropped tables, structure recognition is approached either by decoding markup, as in EDD [9] and TableFormer [10], or by detecting rows and columns, as in the Table Transformer [2]. ConRTF [12] showed that the D-FINE detector [11] beats a coordinate-regression real-time DETR on cropped tables (99.29 vs. 98.90 GriTS-Con on PubTables-1M), and Docling's layout models include D-FINE variants [13]. We ask whether such a detector, trained from an ImageNet backbone without a released table model, raises exact page-level extraction.

Our contributions are: (i) AE3-TL, which improves on the previous best model [6] on all six PubTables-v2 table metrics, with paired bootstrap intervals against a reproduced baseline; (ii) a training recipe, made of crop and page stages plus a targeted caption-linking stage, and an inference stack whose vectorised post-processor helps make per-page processing 1.73× faster; and (iii) a density and structural analysis indicating that the gain comes mainly from getting rows and columns right, together with our development findings (Appendix A).

[1] We compare against POTATR-v1.1-Pub, released by its authors with code and weights (MIT licence) [6]. Unless marked as published, POTATR numbers are from our re-run of these weights with the official scorer.

2 Task and metrics

A page p carries ground-truth tables $\mathcal{T}_p$, each a grid of rows, columns, spanning cells and headers with a text in every cell, and optionally a caption and a footer; a model outputs tables $\hat{\mathcal{T}}_p$. On each page, every metric pairs predicted and ground-truth tables one-to-one by Hungarian matching that maximises its summed score [1]. GriTS [3] compares a ground-truth grid $\mathbf{A}$ with a predicted grid $\mathbf{B}$:

$$\mathrm{GriTS}_f(\mathbf{A},\mathbf{B}) = \frac{2\sum_{i,j} f(\tilde{\mathbf{A}}_{i,j}, \tilde{\mathbf{B}}_{i,j})}{|\mathbf{A}| + |\mathbf{B}|}, \qquad (1)$$

where $\tilde{\mathbf{A}}, \tilde{\mathbf{B}}$ are the most similar two-dimensional substructures of $\mathbf{A}$ and $\mathbf{B}$, $|\mathbf{A}|$ counts the grid cells of $\mathbf{A}$, and $f \in [0,1]$ compares two grid cells by their row-column spans (Top) or by the normalised longest common subsequence of their texts (Con). TEDS [9] is $1 - \mathrm{EditDist}(T_a, T_b)/\max(|T_a|, |T_b|)$ over HTML trees; TEDS-S ignores the text. Over a dataset, each metric is an F1 score: its true-positive score (summed cell similarity for GriTS, TEDS for TEDS) is summed over the matched pairs M of all pages, and precision and recall divide it by the predicted and ground-truth totals, in grid cells for GriTS and tables for TEDS [1], so a missed or spurious table counts as zero. *Exact table match* [2, 1] credits only tables reproduced exactly:

$$\mathrm{Acc}_f = \frac{1}{|\mathcal{T}|} \sum_{(\mathbf{A},\mathbf{B})\in M} \mathbf{1}\big[\mathrm{GriTS}_f(\mathbf{A},\mathbf{B}) = 1\big], \qquad (2)$$

where $\mathcal{T}$ collects the ground-truth tables of all pages. $\mathrm{Acc}_{\mathrm{Con}}$ thus needs the text of every grid position right, and $\mathrm{Acc}_{\mathrm{Top}}$ the span of every cell. All scores come from the benchmark's official scorer (Section 4).

The two families respond very differently to small per-cell differences. GriTS and TEDS give partial credit, so a lower per-cell error rate moves them roughly linearly. Exact match is a product over cells: if each of a table's n cells fails independently with probability ε, then $\mathrm{Acc}(n) = (1-\varepsilon)^n \approx e^{-\varepsilon n}$, so the same improvement compounds with table size. Real errors cluster in rows and columns, so Section 6 fits an effective rate to data.

3 Method

Table 1 separates what we adopt from prior work, including the previous best model's task formulation [6], from what is ours.

Table 1: What AE3-TL adopts from prior work and what is ours.

Component	Status
Task, 16 classes, output schema [6]	Adopted
Relation head (shape, loss, weight) [8, 6]	Adopted
Averaging both link directions [16]	Adopted
Crop-then-page training order [2, 6]	Adopted
10-epoch pilot schedule (10, ×0.66) [6]	Adopted (values)
Text-extraction functions [16]	Adopted
D-FINE detector, matcher and code base [11]	Adopted, extended
Crop stage trained from an ImageNet backbone	Ours
Relation loss inside D-FINE's criterion	Ours
Class growth by name (6 → 16)	Ours
Linking stage: softmax loss, reweighting, oversampling	Ours
Single-best-table linking	Ours
Vectorised post-processor (TATR's logic)	Reimplemented
Parity-gated ONNX package	Ours

Detector. A D-FINE-L detector (HGNetv2-B4 backbone, 768×768 input, 300 queries) predicts objects in 16 classes: tables and rotated tables, rows and columns, column headers and projected row headers, spanning cells, and captions and footers. Rather than regressing box edges directly, D-FINE predicts at each decoder layer l a distribution $\Pr^l(n)$ over offsets and sets the edge distances to $\mathbf{d}^l = \mathbf{d}^0 + \{H, H, W, W\} \cdot \sum_{n=0}^{N} W(n) \Pr^l(n)$, where $\mathbf{d}^0$ holds the initial box's edge distances, H and W are its height and width, and $W(n)$ is a non-uniform weighting function (Eq. 2 of [11]). The model has 31.6M parameters, against 28.9M for the previous best model [6] counted the same way.

Relations. Let $e_u \in \mathbb{R}^{256}$ be the last decoder layer's embedding of query u (denoising queries excluded). A multilayer perceptron (MLP) h with two hidden layers of 512 ReLU units (0.53M parameters) scores the ordered pair "u is a child of v" as $s(u,v) = h([e_u; e_v])$. The page stage adds a pairwise term to D-FINE's criterion:

$$\mathcal{L} = \mathcal{L}_{\text{D-FINE}} + \lambda\, \mathcal{L}_{\mathrm{rel}}, \quad \lambda = 0.05, \qquad (3)$$

$$\mathcal{L}_{\mathrm{rel}} = \frac{1}{|\mathcal{B}|} \sum_{p\in\mathcal{B}} \frac{1}{|\mathcal{P}_p|} \sum_{(u,v)\in\mathcal{P}_p} \ell\big(s(u,v), y_{uv}\big). \qquad (4)$$

Here $\mathcal{B}$ holds the batch's pages with relations and ℓ is binary cross-entropy on logits. $\mathcal{P}_p$ pairs every query v matched to a table, caption or footer with every other query u, plus 512 random pairs, and $y_{uv} = 1$ iff the ground truth links the objects that D-FINE's Hungarian matching [7, 11] assigns to u and v. A final one-epoch linking stage raises λ to 0.30 and adds a softmax over each page's tables for every caption or footer $c \in C$ with true table t_c:

$$\mathcal{L}_{\mathrm{soft}} = -\frac{1}{|C|} \sum_{c\in C} \log \frac{\exp s(c, t_c)}{\sum_{t\in\mathcal{T}_p} \exp s(c,t)}. \qquad (5)$$

At inference, c goes to

$$\hat{t}(c) = \arg\max_t \ \tfrac{1}{2}\big[\sigma(s(c,t)) + \sigma(s(t,c))\big], \qquad (6)$$

with σ the logistic function, provided the score clears a floor τ. The head's shape, loss and weight and the

averaging of the two directions are adopted (Table 1); the softmax term and the assignment to a single best table are ours. Structure objects are grouped geometrically; Appendix D gives the remaining settings.

Post-processing. Our vectorised reimplementation of TATR's grid construction [2] gives identical tables on 50 fixed pages (2,951 cells), with 7.3× faster post-processing. In a larger check, 187 of 703,374 cells differ, and only in the order of their row or column indices. Cell text comes from the word layer through the benchmark's own functions.

4 Training and evaluation

Training. We use D-FINE's public code base with our extensions, on 8×H100 at batch 256. The crop stage ran 28 epochs on 758,849 tight table crops from PubTables-1M (17.8 h over three runs; 2.5 h of epochs after a divergence were discarded). The page stage ran 10 epochs on 408,647 PubTables-v2 Single Pages (6.8 h), with the epoch count and decay of POTATR's 10-epoch pilot (its Table 1) and our own optimizer settings. The linking stage ran one epoch (0.5 h). The kept runs total about 200 H100-hours (about 220 with the discarded epochs) and about 26M image passes. The previous best model's schedule used about 55.6M passes by its paper (100 + 35 epochs) or 54.4M by its released code (97 + 35) [6, 16], excluding the pretraining of its initial table model. For reference, that model's own 10-epoch setting reached GriTS-Con 0.892, with its authors' code on 8×T4 at batch 16 (split not stated) [6]. After the same epoch count and decay, and before linking, AE3-TL reaches 0.978 on test. This compares our run with a published number, not with a re-run under identical conditions.

Selection. All weights and thresholds were chosen on validation data, and the raw-versus-EMA rule was fixed before any test scoring. The test set was scored about ten times, for released versions and package checks such as the caption-text switch and the batch-size test (Appendices A, D). Crop-stage thresholds were tuned on a PubTables-1M test subset, which moves GriTS by 0.0004 on true validation tables.

Protocol. We score the shipped ONNX package on the PubTables-v2 Single Pages test set (7,680 pages, 9,047 tables), with dataset words and the official scorer [16]. As the baseline, we re-ran the released POTATR weights [6]; the re-run matches the published scores within 0.0022 on all six metrics. For a metric S, we report the paired gain $\hat{\Delta} = S_{\text{ours}}(\mathcal{D}) - S_{\text{base}}(\mathcal{D})$ on the test pages $\mathcal{D}$. For the interval, we resample $\mathcal{D}$ with replacement R=2,000 times, recompute the gain Δ_r^* on each resample, and take its 2.5th and 97.5th percentiles [14]. Speed is measured on one A10 at batch 1, at the benchmark's timing boundary. An article-level audit, validated with planted duplicates, finds no overlap between the test articles and any of our training data.

Table 2: PubTables-v2 Single Pages test set, dataset words. "POTATR" is our re-run of the released weights [6]. Paired gain: AE3-TL minus POTATR on the same pages. Full benchmark in Appendix B.

Metric	POTATR	AE3-TL	Paired gain [95% CI]
GriTS Top	0.9674	**0.9788**	+0.0114 [.0098, .0131]
GriTS Con	0.9647	**0.9769**	+0.0122 [.0104, .0140]
Acc Top	0.7013	**0.7962**	+0.0948 [.0860, .1030]
Acc Con	0.6447	**0.7367**	+0.0920 [.0836, .0999]
TEDS-S	0.9577	**0.9703**	+0.0126 [.0107, .0147]
TEDS	0.9516	**0.9654**	+0.0138 [.0116, .0160]

5 Results

AE3-TL leads on every metric, and every interval excludes zero (Table 2). The averaging metrics move by 1.1 to 1.4 points, while exact match moves by 9.2 to 9.5 points: 26% fewer tables contain an error.

Speed and cost. On one A10, AE3-TL processes 80,808 pages per hour (44.6 ms each) against 46,753 (77.0 ms) for POTATR [6]: 1.73× faster, or $30 against $52 per million pages at the benchmark's pricing.

Captions and footers. Footer text F1 is higher (0.9849 vs. 0.9793; +0.0056 [0.0019, 0.0094]), and caption text F1 slightly lower (0.9766 vs. 0.9807; −0.0041 [−0.0080, −0.0005]). Captions are detected reliably (recall 99.6%, precision 99.1% in an earlier pass). On 2,000 validation pages, linking brings caption F1 within 0.002 of an oracle with ground-truth links (Appendix A). The residual therefore most likely lies in box extents.

6 Why the gain compounds

We test the compounding model of Section 2 on test pages binned by ground-truth cell count.[2] We fit $\log \text{Acc}(n) = \log c - \varepsilon n$ by weighted least squares, weighting each bin by its table count. Over the seven bounded bins, the fit gives effective rates $\varepsilon_{\text{POTATR}} = 3.30 \times 10^{-3}$ and $\varepsilon_{\text{AE3}} = 1.90 \times 10^{-3}$ per cell (R^2 of 0.96 and 0.92), a ratio of 0.58. Dropping any one bin gives 0.55–0.63, and adding the open >250 bin at $n \in [300, 350]$ gives 0.52–0.53. Equivalently, exact match halves every $\ln 2/\varepsilon \approx 210$ cells for POTATR and every ≈ 365 cells

[2]The density bins and the structural statistics in this section come from an evaluation pass made before a final word-handling fix. That pass scored 0.731 overall exact content match, against 0.737 in Table 2; POTATR's bins reproduce its headline exactly.

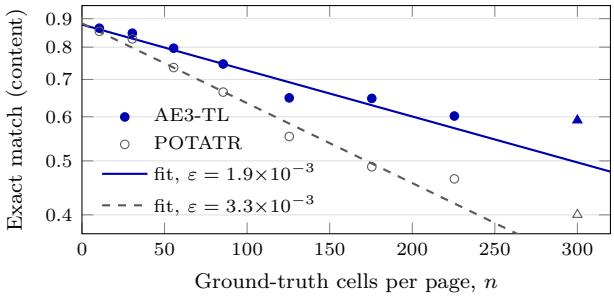


Figure 1: Exact content match against page density (log scale), with fits $ce^{-\varepsilon n}$ to the seven bounded bins (circles). Triangles show the open >250 bin, placed at n=300. The intercepts agree (0.878 for AE3-TL, 0.883 for POTATR), while AE3-TL's decay rate is 0.58× POTATR's.

for AE3-TL. The exponential form fits better than linear or power-law decay (Appendix C). The intercepts are close, $c_{\text{POTATR}} = 0.883$ against $c_{\text{AE3}} = 0.878$, and stay between 0.86 and 0.90 across the leave-one-out fits. The two systems thus make size-independent errors at about the same rate, and AE3-TL cuts the size-dependent part by roughly 40–45%. Here ε is an effective rate per cell on the page, not a literal independent probability. This is why the gap widens from 1 point on sparse pages to 19 points above 250 cells (Fig. 1; Appendix C).

Direct structural statistics point the same way. Table detection is on par: the table count is right on 99.43% of pages for AE3-TL, against 99.51%. Boundaries are not sharper, with a median edge distance of 6.17 against 5.65 px. But the grid size is right more often: AE3-TL predicts both the row and column counts exactly on 88.4% of tables, against 85.1%. The gain is thus one of getting the set of rows and columns right, not of locating tables or edges more precisely. Which part of the detector produces it is open. D-FINE's distribution-based box regression is a natural suspect [12], but our boundaries are not tighter, so settling it needs an ablation.

7 Lessons and limitations

Real PDFs. The lead carries over to words extracted from real PDFs. We ran a pilot on 68 validation pages (89 tables), which are easier than the test set as a whole. On them, AE3-TL reproduces 87.6% of tables exactly, against 74.2% for POTATR under the same conditions. Because the benchmark builds its reference text from its own word files, text from any other extractor is marked wrong for spacing and tokenisation even when it is correct, so we rebuild each reference from the words the system receives, with the benchmark's own code; this rebuild reproduces the official references at 98.94% exact match.

Scope and next steps. Our headline results use the benchmark's word layer on scientific articles, and the real-PDF evidence is so far a pilot. We have not yet isolated the detector's contribution with an ablation, all results come from one training run, and we have not compared training cost with the previous best model on the same hardware (Appendix D). The next experiments are that ablation, an out-of-domain test on financial tables, a full real-PDF test set, and a same-hardware training-cost comparison.

Release. Our inference code, including the vectorised post-processor, and the version-3 ONNX weights are released under the Apache License 2.0 (links on page 1; hashes in Appendix E).

References

[1] B. Smock et al. PubTables-v2: A new large-scale dataset for full-page and multi-page table extraction. arXiv:2512.10888, 2025.

[2] B. Smock, R. Pesala, R. Abraham. PubTables-1M: Towards comprehensive table extraction from unstructured documents. In CVPR, 2022. arXiv:2110.00061.

[3] B. Smock, R. Pesala, R. Abraham. GriTS: Grid table similarity metric for table structure recognition. In ICDAR, 2023. arXiv:2203.12555.

[4] A. Nassar et al. SmolDocling: An ultra-compact vision-language model for end-to-end multi-modal document conversion. arXiv:2503.11576, 2025.

[5] H. Wei, Y. Sun, Y. Li. DeepSeek-OCR: Contexts optical compression. arXiv:2510.18234, 2025.

[6] B. Smock et al. POTATR: A lightweight image-to-graph model for page-level table extraction. In Document Analysis Systems (DAS), 2026. arXiv:2606.09788.

[7] N. Carion et al. End-to-end object detection with transformers. In ECCV, 2020. arXiv:2005.12872.

[8] S. Shit et al. Relationformer: A unified framework for image-to-graph generation. In ECCV, 2022. arXiv:2203.10202.

[9] X. Zhong, E. ShafieiBavani, A. Jimeno Yepes. Image-based table recognition: Data, model, and evaluation. In ECCV, 2020. arXiv:1911.10683.

[10] A. Nassar et al. TableFormer: Table structure understanding with transformers. In CVPR, 2022. arXiv:2203.01017.

[11] Y. Peng et al. D-FINE: Redefine regression task in DETRs as fine-grained distribution refinement. In ICLR, 2025. arXiv:2410.13842.

[12] E. Thomas et al. ConRTF: Edge-constrained boundary distribution refinement for realtime transformer table structure recognition. In ICDAR, 2026. arXiv:2607.00734.

[13] N. Livathinos et al. Advanced layout analysis models for Docling. arXiv:2509.11720, 2025.

[14] B. Efron, R. J. Tibshirani. An Introduction to the Bootstrap. Chapman & Hall, 1993.

[15] J. Deng et al. ImageNet: A large-scale hierarchical image database. In CVPR, 2009.

[16] Kensho Technologies. POTATR: code, weights and page-level evaluation for PubTables-v2, version 0.2.0. https://github.com/kensho-technologies/POTATR, 2026.

[17] Y. Li et al. dots.ocr: Multilingual document layout parsing in a single vision-language model. arXiv:2512.02498, 2025.

[18] Google DeepMind. Gemini 3.1 Pro model card. https://storage.googleapis.com/deepmind-media/Model-Cards/Gemini-3-1-Pro-Model-Card.pdf, February 2026.

A Development findings

The measured steps behind the model, in the order we took them, including the ones we rejected. Each changed one thing and was measured before the next, on validation data unless stated. This records the path we took; it is not an ablation. Three of our largest fixes raised no error: a converter that dropped 91,554 relation edges, a word-assignment bug that capped the pipeline at 0.850 GriTS-Con even with ground-truth boxes (0.986 after the fix), and index-based class mapping (cap 0.959). Ground-truth ceiling checks and export parity gates caught all three.

Table 3: Development findings. Scores are GriTS-Con unless stated; "val" is validation data.

Step	What we measured	Decision
Crop framing	Our first crops carried about 36 px of padding, against $\leq$5 px in TATR's training data. The same model scored 0.880 on padded crops and 0.986 on tight crops.	Rebuilt 758,849 tight crops
Harness check	After a switch to macro averaging, TATR-v1.1 [19] reproduces at 0.9854, against its published 0.9855.	Harness trusted
Data conversion	The page converter dropped 91,554 caption and footer relation edges. After the fix, 100% of 10,741 child objects are covered (400 val pages).	Fixed before training
Word assignment	With ground-truth boxes, our pipeline's ceiling was 0.850. Fixing word-to-cell assignment raised it to 0.986 (1,959 val pages).	Kept
Class mapping	Index-based head mapping caps the pipeline at 0.959, against 0.999 when mapping by name (51,214 val pages).	Map by name
Crop learning rate	At 7e-4 the run diverged (val mAP 0.943 $\rightarrow$ 0.612). We restarted from epoch 4 at 2.5e-4 and completed 28 epochs.	2.5e-4
Crop-stage result	0.9908 vs. TATR-v1.1 0.9854 on tight crops of all 93,834 PubTables-1M test tables, same harness.	Initialises page stage
Resolution	Rows 6–8 px tall hold 0.9% of cells. Training at 1024 px would gain about 0.2 points for about 1.8$\times$ the compute.	Stayed at 768 px
Checkpoint rule	Raw beats EMA weights at epoch 9 by +0.00065 [0.00039, 0.00093] (all 51,214 val pages, rule registered before test).	Raw for v1
Caption diagnosis	On the full validation set, caption F1 is 0.835 with our boxes and geometric links, and 0.965 with ground-truth links. The relation head trained at loss weight 0.05 reaches only 0.808.	Linking is the gap
Linking stage (v3)	On a 2,000-page validation subset, caption F1 is 0.970, against 0.841 with geometric links and 0.972 with ground-truth links. On test it confirms at +0.098 [0.090, 0.106], with GriTS-Con $-$0.0011 [$-$0.0020, $-$0.0003].	Released as v3
Shipped package	The argmax linking rule recovers 268 captions and 162 footers that the per-pair threshold missed (val). Using the benchmark's caption-text functions takes test caption F1 from 0.934 to 0.977.	All kept
Export	ONNX vs. PyTorch: 17 of 137,687 cells differ, with no table-count differences (2,000 val pages). Batch-16 inference changes 322 of 9,062 predicted test tables.	ONNX at batch 1
Real PDFs	Adding block, line and span numbering to extracted words changes scores by $\leq$0.0004. Rebuilding the reference text explains the gap (Section 7).	Rebuild references

B Full benchmark tables

Rows marked ‡ are ours; all other rows are as published [6].

Table 4: Page-level table extraction, PubTables-v2 Single Pages test set.

Model	GriTS Top	GriTS Con	Acc Top	Acc Con	TEDS-S	TEDS
SmolDocling-256M [4]	0.2502	0.2177	0.2800	0.0298	0.5980	0.5250
GraniteDocling-258M [22]	0.7252	0.6670	0.3794	0.0829	0.6685	0.6155
Qwen2.5-VL-3b [20]	0.4672	0.3803	0.3651	0.0619	0.2859	0.2632
Granite-Vision-3.2-2b [21]	0.8015	0.7480	0.4245	0.0331	0.8665	0.8092
DeepSeek-OCR [5]	0.8629	0.8207	0.6184	0.1671	0.7996	0.7639
DeepSeek-OCR 2 [23]	0.8839	0.8386	0.5258	0.1120	0.7688	0.7263
dots.ocr [17]	0.9241	0.8991	0.6425	0.1872	0.9168	0.8743
Claude Opus 4.6 [24]	0.9146	0.9106	0.4934	0.2713	0.9073	0.8945
GPT-5.4 [25]	0.9208	0.9102	0.5461	0.2074	0.9084	0.8874
Gemini 3.1 Pro [18]	0.9500	0.9418	0.6348	0.3524	0.9392	0.9250
POTATR + EasyOCR [26]	0.9555	0.7819	0.6595	0.0022	0.9468	0.8162
POTATR + PaddleOCR [27]	0.9607	0.8758	0.6915	0.0074	0.9528	0.8759
POTATR + docTR [28]	0.9601	0.8799	0.6707	0.0683	0.9500	0.8952
POTATR + DT	0.9665	0.9636	0.6992	0.6454	0.9565	0.9502
POTATR + DT (our re-run)‡	0.9674	0.9647	0.7013	0.6447	0.9577	0.9516
AE3-TL + DT (ours)‡	**0.9788**	**0.9769**	**0.7962**	**0.7367**	**0.9703**	**0.9654**

Table 5: Caption and footer text, character-level LCS F1 (the benchmark paper does not report footers).

Model	Caption F1	Caption P	Caption R	Footer F1
Claude Opus 4.6	0.9611	0.9866	0.9370	–
GPT-5.4	0.9658	0.9743	0.9575	–
Gemini 3.1 Pro	0.9774	0.9882	0.9669	–
POTATR + EasyOCR	0.9789	0.9830	0.9749	–
POTATR + PaddleOCR	0.9764	0.9799	0.9728	–
POTATR + docTR	0.9786	0.9828	0.9744	–
POTATR + DT	0.9781	0.9817	0.9746	–
POTATR + DT (our re-run)‡	0.9807	0.9824	0.9790	0.9793
AE3-TL + DT (ours)‡	0.9766	0.9756	0.9776	**0.9849**

Table 6: Speed and cost. Our rows: one NVIDIA A10 (the paper used an A10G), batch 1, model forward pass plus post-processing, file I/O excluded, priced at $2.45 per hour.

Model	Params	Pages/hour	$ per 1M pages
Claude Opus 4.6	–	244	36,680
GPT-5.4	–	326	16,800
Gemini 3.1 Pro	–	393	15,000
Qwen2.5-VL-3b	3B	386	6,336
dots.ocr	3B	572	4,281
DeepSeek-OCR 2	3B	1,201	2,038
SmolDocling-256M	256M	1,247	1,963
GraniteDocling-258M	258M	1,847	1,325
POTATR (published, A10G)	29M	52,031	47
POTATR (our re-run, A10)‡	29M	46,753	52
AE3-TL (ours, A10)‡	32M	**80,808**	**30**

C Accuracy by page density

These figures come from the earlier evaluation pass described in the footnote to Section 6. Fitting the seven bounded bins, the exponential form $c\,e^{-\varepsilon n}$ has a weighted squared error of 3.8 for POTATR and 4.4 for AE3-TL. A linear fit gives 8.3 and 5.8, and a power law 26.9 and 11.2.

Table 7: Accuracy by ground-truth cells per page on the test set. POTATR is our re-run of its released weights [6]. The first and last rows give the exact-match figures quoted in the abstract.

			GriTS-Con		TEDS		Exact match (Acc Con)	
Cells per page	Pages	Tables	POTATR	AE3-TL	POTATR	AE3-TL	POTATR	AE3-TL
1–20	625	631	0.9617	0.9656	0.9579	0.9600	0.8542	0.8653
21–40	1,243	1,313	0.9734	0.9747	0.9662	0.9664	0.8279	0.8477
41–70	1,570	1,776	0.9717	0.9790	0.9589	0.9696	0.7354	0.7967
71–100	1,232	1,478	0.9722	0.9817	0.9555	0.9686	0.6644	0.7463
101–150	1,329	1,631	0.9663	0.9792	0.9480	0.9672	0.5530	0.6487
151–200	848	1,086	0.9644	0.9764	0.9422	0.9641	0.4880	0.6473
201–250	401	530	0.9650	0.9788	0.9411	0.9659	0.4642	0.6019
>250	432	602	0.9500	0.9696	0.9188	0.9535	0.4003	0.5914

D Training settings

Crop stage. AdamW [29] with weight decay 1.25×10^{-4}, 640×640 input, 300 queries and a batch size of 256 (8 GPUs × 32). The HGNetv2-B4 backbone [30] starts from ImageNet weights [15]. Epochs 0–1 ran at a main learning rate of 7×10^{-4} (backbone 5×10^{-6}) and epochs 2–4 at 7×10^{-4} (backbone 3.5×10^{-5}), without warm-up. That run's validation mAP collapsed in epochs 5–6. Training resumed from the epoch-4 checkpoint at 2.5×10^{-4} (backbone 1.5×10^{-5}), with step decay ×0.1 at epochs 22 and 26, through epoch 27. Photometric distortion ($p = 0.5$) and zoom-out ($p = 0.3$, up to 1.2×) are applied until epoch 22. We keep EMA weights.

Page stage. AdamW at 2.5×10^{-4} (backbone 1.25×10^{-5}), weight decay 1.25×10^{-4}, 1,000 warm-up iterations and step decay, so the learning rate in epoch e is $\eta_e = \eta_0\,\gamma^{\lfloor e/2 \rfloor}$ with $\eta_0 = 2.5 \times 10^{-4}$ and $\gamma = 0.66$ (decays at epochs 2, 4, 6 and 8). Input is 768×768 with a batch size of 256, for 10 epochs, initialised from the crop stage's EMA weights. Augmentation is photometric distortion ($p = 0.3$) and zoom-out ($p = 0.3$, up to 1.1×). The relation loss has weight 0.05 with 512 sampled negative pairs per page.

Class growth by name. The crop stage has 6 classes and the page stage 16, and our two converters order them differently. Every classification layer therefore copies rows by name: $W^{\text{page}}_k = W^{\text{crop}}_{\pi(k)}$ and $b^{\text{page}}_k = b^{\text{crop}}_{\pi(k)}$ for shared classes k, where π matches class names; the ten new classes are initialised afresh. Index mapping would have swapped rows and columns.

Linking stage. The linking stage trains one epoch from the epoch-9 raw weights. The detector's learning rate is 2.5×10^{-5} (backbone 1.25×10^{-6}), and the relation head has its own group at 10^{-3}. The relation weight rises from 0.05 to 0.30, and the softmax term $\mathcal{L}_{\text{soft}}$ is added. Multi-table pages are oversampled 3× and rotated pages 2×, and the caption and footer classes are weighted 1.5×. EMA is kept, and the released weights are the EMA after this epoch.

Inference. We run at batch 1; batch 16 changes 322 of 9,062 predicted test tables. The model is exported as ONNX [31] opset 17 in fp32.

Training budget. Our kept runs used about 26M image passes and about 100K optimizer steps at batch 256, or about 200 H100-hours (about 220 with the discarded crop epochs). The previous best model's schedule implies about 55.6M image passes and 3.47M steps at batch 16 on 8×T4 by its paper (100 + 35 epochs), or 54.4M and 3.40M by its released code (97 + 35) [6, 16], plus the pretraining of its initial table model. Image passes and steps are not a measure of compute: the two detectors cost different amounts per image and were trained on different GPUs. A like-for-like comparison needs both models timed on the same hardware, which we leave to future work.

E Evaluated artefacts

The hashes identify the exact model evaluated in this paper.

File	sha256
Checkpoint (v3, EMA after linking)	`47000f3b785b65c4685e283e266ed67be0eccc2018abef8b183e5f903e34f48c`
Page model (ONNX)	`91f5183a8b4a2df2afc80992c89cc6df30456bb6d5a7dfa19d2cb26414cfdcff`
Relation head (ONNX)	`5867fa72915d47b03942b94f0ab4d73946a7543790e95560882536e9f94011c7`

The page model takes float32 images $[B, 3, 768, 768]$, RGB in $[0, 1]$. It outputs logits $[B, 300, 16]$, boxes $[B, 300, 4]$ (normalised c_x, c_y, w, h), query embeddings and encoder indices. The relation head takes query embeddings and index pairs $[n, 2]$ and returns edge logits $[n]$.

Additional references for the appendices

[19] B. Smock, R. Pesala, R. Abraham. Aligning benchmark datasets for table structure recognition. In ICDAR, 2023. arXiv:2303.00716.

[20] S. Bai et al. Qwen2.5-VL technical report. arXiv:2502.13923, 2025.

[21] Granite Vision Team. Granite Vision: a lightweight, open-source multimodal model for enterprise intelligence. arXiv:2502.09927, 2025.

[22] IBM Granite. Granite-Docling-258M. https://huggingface.co/ibm-granite/granite-docling-258M, 2025.

[23] H. Wei, Y. Sun, Y. Li. DeepSeek-OCR 2: Visual causal flow. arXiv:2601.20552, 2026.

[24] Anthropic. System card: Claude Opus 4.6. https://www-cdn.anthropic.com/0dd865075ad3132672ee0ab40b05a53f14cf5288.pdf, February 2026.

[25] OpenAI. GPT-5.4 Thinking system card. https://deploymentsafety.openai.com/gpt-5-4-thinking, 2026.

[26] JaidedAI. EasyOCR. https://github.com/JaidedAI/EasyOCR, 2024.

[27] C. Cui et al. PaddleOCR 3.0 technical report. arXiv:2507.05595, 2025.

[28] Mindee. docTR: Document text recognition. https://github.com/mindee/doctr, 2021.

[29] I. Loshchilov, F. Hutter. Decoupled weight decay regularization. In ICLR, 2019. arXiv:1711.05101.

[30] PaddlePaddle Authors. PaddleClas: PP-HGNetV2. https://github.com/PaddlePaddle/PaddleClas.

[31] ONNX Community. ONNX: Open Neural Network Exchange. https://github.com/onnx/onnx.